

BEST REVIEWER 양성과정

Refactoring 연습 #1

GildedRose — VSCode Java & C++ 개발자 과정

★ C++ (VSCode + clangd) 위주 · Java (VSCode + Extension Pack) 참고 대응

C++17

clangd

Google Test

CMake

Java 참고: Java 17+

Java 참고: JUnit 5

Java 참고: Extension Pack for Java

1
테스트 먼저

2
코드 정리

3
조건 분리

4
메서드 추출

5
클래스 분리

6
추상화

7
기능 추가

Refactoring 실습 #1 — 목적 & 요구사항



테스트 보호막 구축

레거시 코드에 회귀 테스트(Regression Test) 먼저 작성 → 리팩토링 안전망 확보



구조 재설계

Replace Type-code with Subclasses + Template Method Pattern 적용



기능 추가 (F&B)

Food & Beverage 아이템 추가 가능한 구조로 리팩토링 → 새 클래스 추가



프로젝트 설정

- TDD 과정에서 작성한 GildedRose 저장소 그대로 사용
- 기존 PR은 merge 하지 않은 상태에서 리팩토링용 branch 새로 생성 (예: refactoring)
- C++: CMakeLists.txt + Google Test 환경 | Java: Maven + JUnit 5 환경
- VSCode Terminal → git checkout -b refactoring dev

핵심 원칙: 기능이 변하지 않는다는 것을 증명하는 테스트 코드 없이 절대 리팩토링하지 마세요!

GildedRose — 프로젝트 소개 & 비즈니스 규칙

📄 배경 스토리

- GildedRose 는 가상의 판매점 시스템입니다.
- 기존 개발자가 만든 legacy updateQuality() 함수를 당신이 인수받았습니다.
- 요구사항: Food & Beverage(F&B) 아이템 추가 기능 구현
- 문제: 중첩 if 문으로 새 아이템 추가가 매우 어렵습니다.
- 목표: 먼저 테스트를 작성하고, 리팩토링 후 기능을 추가합니다.

일반 아이템

매일 quality -1, sellIn -1
sellIn 지나면 quality -2

Aged Brie

오래될수록 quality ↑
sellIn 지나면 quality +2

Sulfuras

quality 항상 80 고정
sellIn/quality 변화 없음

Backstage Pass

sellIn<11: +2, sellIn<6: +3
기한 후 quality = 0

Conjured (미구현)

일반의 2배 감소 (신규 추가)
F&B 아이템과 함께 추가

❌ 레거시 코드의 4가지 문제점

· 가독성 최악: 중첩 if 5단계

· OCP 위반: 새 아이템 추가 시 전체 if 수정

· 타입 코드 남용: 문자열 비교로 타입 분기

· 테스트 불가: 로직이 뒤섞여 단위 테스트 불가

GildedRose — 프로젝트 소개 & 비즈니스 규칙

C++ (VSCode + clangd)

```
// C++ 레거시 updateQuality() - 전체가 한 함수에!
void GildedRose::updateQuality() {
    for (int i = 0; i < items.size(); i++) {
        if (items[i].name != AGED_BRIE &&
            items[i].name != BACKSTAGE_PASS) {
            if (items[i].quality > 0) {
                if (items[i].name != SULFURAS)
                    items[i].quality--;
            }
        } else {
            if (items[i].quality < 50) {
                items[i].quality++;
                if (items[i].name == BACKSTAGE_PASS) {
                    if (items[i].sellIn < 11)
                        if (items[i].quality < 50)
                            items[i].quality++;
                    if (items[i].sellIn < 6)
                        if (items[i].quality < 50)
                            items[i].quality++;
                }
            }
        }
        // sellIn 감소 ... 또 조건문 ...
    }
}
```

리팩토링 로드맵 & Template Method Pattern

1

테스트 먼저

Google Test / JUnit 5로 회귀 테스트

2

코드 정리

상수 추출, 변수 추출, 중복 조건 제거

3

조건 분리

if(!~) 반전, else-if 병합, 조건 단순화

4

메서드 추출

타입별 updateQuality*() 메서드 분리

5

클래스 분리

4개 아이템 클래스 생성, 메서드 이동

6

추상화

GildedRoseItem 추상 클래스 + 다형성

7

기능 추가

F&B 아이템 새 클래스로 추가

각 단계마다 테스트를 실행해 기존 동작이 유지되는지 확인하세요! (VSCode: Ctrl+Shift+P → Run All Tests)

리팩토링 로드맵 & Template Method Pattern



Template Method Pattern — 핵심 설계 원칙

C++ (VSCode + clangd)

```
// C++ - Template Method Pattern 적용 전 (타입 코드)
class Employee {
public:
    static const int ENGINEER = 0, MANAGER = 1, INTERN = 2;
    Employee(int type) : type_(type) {}
    double getPay() {
        switch (type_) {
            case ENGINEER: return 5000;
            case MANAGER: return 8000;
            case INTERN: return 2000;
            default: throw std::invalid_argument("Unknown type");
        }
    }
private:
    int type_;
};

// C++ - Template Method Pattern 적용 후 (다형성)
class Employee {
public:
    virtual double getPay() const = 0; // 추상 메서드 (Template)
    virtual ~Employee() = default;
};
class Engineer : public Employee {
public: double getPay() const override { return 5000; }
};
class Manager : public Employee {
public: double getPay() const override { return 8000; }
};
class Intern : public Employee {
public: double getPay() const override { return 2000; }
};
// Factory: std::unique_ptr<Employee> create(const std::string& type);
```

Java 대응 (VSCode / Maven)

```
// Java - Template Method (Before)
class Employee {
    static final int ENGINEER=0,MANAGER=1;
    int type;
    double getPay() {
        switch(type){
            case ENGINEER: return 5000;
            case MANAGER: return 8000;
            default: throw new ...;
        }
    }
}

// Java - Template Method (After)
public abstract class Employee {
    public abstract double getPay();
}
public class Engineer extends Employee {
    @Override
    public double getPay() { return 5000; }
}
public class Manager extends Employee {
    @Override
    public double getPay() { return 8000; }
}
// Factory Method로 객체 생성 관리
public class EmployeeFactory {
    public static Employee create(String t){
        return switch(t) {
            case "ENGINEER" -> new Engineer();
            case "MANAGER" -> new Manager();
            default -> throw new ...;
        };
    }
}
```

1 Step 1 — 테스트 먼저 (회귀 테스트 작성)

리팩토링 전 안전망 구축 — Google Test / JUnit 5

VSCode 단축키

C++: Ctrl+Shift+P → CMake: Run Tests | ctest --test-dir build -V

Java: Ctrl+Shift+P → Java: Run All Tests | mvn test

C++ (VSCode + clangd)

```
// C++ - Google Test 회귀 테스트 (GildedRoseTest.cpp)
#include "GildedRose.h"
#include <gtest/gtest.h>

// ① 일반 아이템 - 매일 quality -1, sellIn -1
TEST(GildedRoseTest, NormalItemDegrades) {
    std::vector<Item> items = {Item("+5 Dexterity Vest", 10, 20)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(9, items[0].sellIn);
    EXPECT_EQ(19, items[0].quality);
}

// ② sellIn 지나면 quality 2배 감소
TEST(GildedRoseTest, NormalItemDegradesTwiceAfterSellIn) {
    std::vector<Item> items = {Item("Normal", 0, 10)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(8, items[0].quality); // 2씩 감소
}
```

Java 대응 (VSCode / Maven)

```
// Java - JUnit 5 회귀 테스트
@Test
void normalItemDegrades() {
    Item[] items = {
        new Item("+5 Dex Vest", 10, 20)};
    GildedRose app = new GildedRose(items);
    app.updateQuality();
    assertEquals(9, items[0].sellIn);
    assertEquals(19, items[0].quality);
}

@Test
void degradesTwiceAfterSellIn() {
    Item[] items = {new Item("x", 0, 10)};
    new GildedRose(items).updateQuality();
    assertEquals(8, items[0].quality);
}

@Test
void qualityNeverNegative() {
    Item[] items = {new Item("x", 5, 0)};
    new GildedRose(items).updateQuality();
    assertEquals(0, items[0].quality);
}
```

Backstage Pass 경계값 테스트도 추가: sellIn=11(+1), sellIn=10(+2), sellIn=5(+3), sellIn=0(quality=0) — 경계값 모두 커버!

1 Step 1 — 테스트 먼저 (회귀 테스트 작성)

C++ (VSCode + clangd)

```
// ㉓ quality는 절대 음수가 되지 않는다
TEST(GildedRoseTest, QualityNeverNegative) {
    std::vector<Item> items = {Item("Normal", 5, 0)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(0, items[0].quality); // 0 유지
}

// ㉔ Aged Brie - 오래될수록 quality 증가
TEST(GildedRoseTest, AgedBrieIncreasesQuality) {
    std::vector<Item> items = {Item("Aged Brie", 5, 10)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(11, items[0].quality);
}

// ㉕ quality는 50을 초과하지 않는다
TEST(GildedRoseTest, QualityNeverMoreThan50) {
    std::vector<Item> items = {Item("Aged Brie", 5, 50)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(50, items[0].quality); // 50 유지
}

// ㉖ Sulfuras - quality/sellIn 변화 없음
TEST(GildedRoseTest, SulfurasNeverChanges) {
    std::vector<Item> items = {Item("Sulfuras, Hand of Ragnaros", 0, 80)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(0, items[0].sellIn);
    EXPECT_EQ(80, items[0].quality);
}
```

Java 대응 (VSCode / Maven)

```
@Test
void agedBrieIncreasesQuality() {
    Item[] items = {
        new Item("Aged Brie", 5, 10)};
    new GildedRose(items).updateQuality();
    assertEquals(11, items[0].quality);
}

@Test
void qualityNeverMoreThan50() {
    Item[] items = {
        new Item("Aged Brie", 5, 50)};
    new GildedRose(items).updateQuality();
    assertEquals(50, items[0].quality);
}

@Test
void sulfurasNeverChanges() {
    Item[] items = {new Item(
        "Sulfuras, Hand of Ragnaros",
        0, 80)};
    new GildedRose(items).updateQuality();
    assertEquals(80, items[0].quality);
}
```


Extract Constant · Extract Variable · Remove Unused Code VSCode 단축키

C++: Ctrl+Shift+R → Extract to variable | Ctrl+. → Extract Constant

Java: Ctrl+Alt+C (상수 추출) | Ctrl+Alt+V (변수 추출) | Alt+Enter → Extract

C++ (VSCode + clangd)

```
// Step 2-1: 매직 스트링을 상수로 추출 (Extract Constant)
// ❌ Before - 문자열 리터럴이 코드에 흩어짐
class GildedRose {
    void updateQuality() {
        if (item.name != "Aged Brie" && item.name != "Backstage passes...") {
            if (item.name != "Sulfuras, Hand of Ragnaros") ...
        }
    }
};

// ✅ After - constexpr 상수로 추출 (오타 방지, 자동 완성 지원)
class GildedRose {
private:
    static constexpr const char* AGED_BRIE = "Aged Brie";
    static constexpr const char* BACKSTAGE_PASS =
        "Backstage passes to a TAFKAL80ETC concert";
    static constexpr const char* SULFURAS = "Sulfuras, Hand of Ragnaros";
    static constexpr int MAX_QUALITY = 50;
    static constexpr int MIN_QUALITY = 0;
public:
    void updateQuality() {
        if (item.name != AGED_BRIE && item.name != BACKSTAGE_PASS) {
            if (item.name != SULFURAS) ...
        }
    }
};
```

Java 대응 (VSCode / Maven)

```
// Java Step 2-1: 상수 추출
// Ctrl+Alt+C → 상수명 입력 → OK
class GildedRose {
    private static final String AGED_BRIE =
        "Aged Brie";
    private static final String BACKSTAGE =
        "Backstage passes to a " +
        "TAFKAL80ETC concert";
    private static final String SULFURAS =
        "Sulfuras, Hand of Ragnaros";
    private static final int MAX_QUALITY = 50;
}
```

C++ (VSCode + clangd)

```
// Step 2-2: 반복되는 items[i] → 변수 추출 (Extract Variable)
// ❌ Before
for (int i = 0; i < items.size(); i++) {
    if (items[i].quality > 0) items[i].quality--;
    // items[i]가 수십 번 반복...
}

// ✅ After - 변수로 추출
for (int i = 0; i < items.size(); i++) {
    Item& item = items[i]; // C++: 참조로 받아야 수정 가능!
    if (item.quality > 0) item.quality--;
    // item 으로 깔끔하게
}
```

Java 대응 (VSCode / Maven)

```
// Java Step 2-2: 변수 추출
// Ctrl+Alt+V → 변수명 입력 → OK
// ❌ Before
for (int i = 0; i < items.length; i++) {
    if (items[i].quality > 0)
        items[i].quality--;
}

// ✅ After
for (int i = 0; i < items.length; i++) {
    Item item = items[i]; // 변수 추출
    if (item.quality > 0)
        item.quality--;
}

// Step 2-3: 불필요한 중첩 제거
// Ctrl+Shift+Delete → Unwrap/Remove
// if (item.quality < 50) {
//     item.quality++; // 이 중첩 제거
// }
```

Invert if · else-if 병합 · 조건 분할 · 불필요 코드 삭제

VSCode 단축키

C++: Ctrl+. → Invert if condition | Ctrl+Shift+R → Simplify

Java: Alt+Enter → Invert if | Alt+Enter → Merge else if

C++ (VSCode + clangd)

```
// Step 3-1: if(!~) → Invert if condition (긍정 조건으로 변경)
// ❌ Before - 이중 부정, 읽기 어려움
if (item.name != AGED_BRIE && item.name != BACKSTAGE_PASS) {
    if (item.quality > 0)
        if (item.name != SULFURAS) item.quality--;
} else { /* Aged Brie / Backstage 로직 */ }

// ✅ After - 긍정 조건으로 읽기 쉬워짐
if (item.name == AGED_BRIE || item.name == BACKSTAGE_PASS) {
    if (item.quality < MAX_QUALITY) item.quality++;
    /* Backstage 추가 로직 */
} else if (item.name == SULFURAS) {
    // 아무것도 안 함
} else {
    if (item.quality > MIN_QUALITY) item.quality--;
}
```

Java 대응 (VSCode / Maven)

```
// Java Step 3-1: Invert if
// Alt+Enter → "Invert 'if' condition"
// ❌ Before
if (!item.name.equals(AGED_BRIE) &&
    !item.name.equals(BACKSTAGE)) {
    // 일반 로직
} else { // Brie/Backstage }

// ✅ After
if (item.name.equals(AGED_BRIE) ||
    item.name.equals(BACKSTAGE)) {
    // Brie/Backstage 로직
} else { // 일반 로직 }
```

조건 단순화 원칙: 긍정 조건 우선 → 부정 조건 최소화 | sellIn 경계: 0 → 1로 변경 (sellIn을 먼저 감소시키면 0이 지난 것을 체크)

C++ (VSCode + clangd)

```
// Step 3-2: else + if → else if 병합 (Merge else if)
// ❌ Before
} else {
    if (item.name == SULFURAS) {
        // 빈 블록
    } else { item.quality--; }
}

// ✅ After
} else if (item.name == SULFURAS) {
    // Sulfuras: 아무것도 안 함
} else {
    if (item.quality > 0) item.quality--;
}

// Step 3-3: 조건 분할 (2개 if 합치기)
// ❌ Before – 같은 타입 조건이 2곳에
if (item.name == AGED_BRIE) { /* sellIn 전 */ }
// ... sellIn 업데이트 ...
if (item.name == AGED_BRIE) { /* sellIn 후 */ }

// ✅ After – 하나로 합치기 (sellIn < 1로 경계 조정)
if (item.name == AGED_BRIE) {
    if (item.quality < MAX_QUALITY) item.quality++;
    if (item.sellIn < 1) // 0 → 1로 변경
        if (item.quality < MAX_QUALITY) item.quality++;
}
```

Java 대응 (VSCode / Maven)

```
// Java Step 3-2: Merge else if
// Alt+Enter → "Merge else if"
// ❌ Before
} else {
    if (item.name.equals(SULFURAS)) {
        // empty
    } else {
        item.quality--;
    }
}

// ✅ After
} else if (item.name.equals(SULFURAS)){
} else {
    if (item.quality > 0) item.quality--;
}

// Java Step 3-3: 2 if 합치기
// ❌ Before (sellIn 전/후 같은 분기 2개)
if(item.name.equals(AGED_BRIE)){...}
// sellIn--
if(item.name.equals(AGED_BRIE)){...}

// ✅ After (하나로 합치기)
if(item.name.equals(AGED_BRIE)){
    if(item.quality < 50) item.quality++;
    if(item.sellIn < 1) // 0→1 변경
        if(item.quality < 50) item.quality++;
}
```

4 Step 4 — 메서드 추출 (Extract Method)

타입별 updateQuality*() 메서드로 분리 + updateSellIn() 분리

VSCode 단축키

C++: 코드 선택 → Ctrl+Shift+R → Extract Function to... | Ctrl+. → Extract function

Java: 코드 선택 → Ctrl+Shift+R → Extract Method | Alt+Enter → Extract Method

✗ Before C++

```
// 하나의 거대한 updateQuality() - 모든 로직이 혼재
void GildedRose::updateQuality() {
    for (int i = 0; i < items.size(); i++) {
        Item& item = items[i];
        if (item.name == AGED_BRIE) {
            if (item.quality < MAX_QUALITY) item.quality++;
            if (item.sellIn < 1)
                if (item.quality < MAX_QUALITY) item.quality++;
        } else if (item.name == BACKSTAGE_PASS) {
            // ... 백스테이지 복잡 로직 ...
        } else if (item.name == SULFURAS) {
            // 아무것도 안 함
        } else {
            if (item.quality > 0) item.quality--;
            if (item.sellIn < 1)
                if (item.quality > 0) item.quality--;
        }
        if (item.name != SULFURAS) item.sellIn--;
    }
}
```

Java 대응 (VSCode / Maven)

```
// Java - Extract Method
// 코드 선택 → Ctrl+Shift+R
// → Extract Method → 메서드명 입력

// ✅ After
public void updateQuality() {
    for (Item item : items) {
        if(item.name.equals(AGED_BRIE))
            updateAgedBrie(item);
        else if(item.name.equals(BACKSTAGE))
            updateBackstage(item);
        else if(item.name.equals(SULFURAS))
            updateSulfuras(item);
        else
            updateNormalItem(item);
        updateSellIn(item);
    }
}

private void updateAgedBrie(Item item){
    if(item.quality < 50) item.quality++;
    if(item.sellIn < 1)
        if(item.quality < 50) item.quality++;
}
```

4 Step 4 — 메서드 추출 (Extract Method)

✓ After C++

```
// ✓ After - 타입별 메서드로 추출
void GildedRose::updateQuality() {
    for (auto& item : items) { // C++11 범위 for
        if (item.name == AGED_BRIE) updateAgedBrie(item);
        else if (item.name == BACKSTAGE_PASS) updateBackstagePass(item);
        else if (item.name == SULFURAS) updateSulfuras(item);
        else updateNormalItem(item);
        updateSellIn(item); // sellIn 업데이트 분리!
    }
}

void GildedRose::updateAgedBrie(Item& item) {
    if (item.quality < MAX_QUALITY) item.quality++;
    if (item.sellIn < 1)
        if (item.quality < MAX_QUALITY) item.quality++;
}

void GildedRose::updateBackstagePass(Item& item) {
    if (item.quality < MAX_QUALITY) item.quality++;
    if (item.sellIn < 11 && item.quality < MAX_QUALITY) item.quality++;
    if (item.sellIn < 6 && item.quality < MAX_QUALITY) item.quality++;
    if (item.sellIn < 1) item.quality = 0;
}

void GildedRose::updateSulfuras(Item&) { /* 변화 없음 */ }
void GildedRose::updateNormalItem(Item& item) {
    if (item.quality > 0) item.quality--;
    if (item.sellIn < 1 && item.quality > 0) item.quality--;
}

void GildedRose::updateSellIn(Item& item) {
    if (item.name != SULFURAS) item.sellIn--;
}
```

Java 대응 (VSCode / Maven)

```
private void updateBackstage(Item item){
    if(item.quality < 50) item.quality++;
    if(item.sellIn<11 && item.quality<50) item.quality++;
    if(item.sellIn<6 && item.quality<50) item.quality++;
    if(item.sellIn<1) item.quality = 0;
}

private void updateSulfuras(Item item){}
private void updateNormalItem(Item item){
    if(item.quality > 0) item.quality--;
    if(item.sellIn<1 && item.quality>0)
        item.quality--;
}

private void updateSellIn(Item item){
    if(!item.name.equals(SULFURAS))
        item.sellIn--;
}
```

AgedBrieItem / BackstagePassItem / SulfurasItem / NormalItem 클래스 생성

VSCode 단축키

C++: Ctrl+Shift+R → Move function to... | Ctrl+N → New File (헤더 생성)

Java: Ctrl+Shift+R → Move to another class | Alt+Shift+T → Change Signature

C++ (VSCode + clangd)

```
// Step 5-1: AgedBrieItem 클래스 생성 (AgedBrieItem.h)
#pragma once
#include "Item.h"

class AgedBrieItem {
    Item& item_; // 참조로 보관
public:
    explicit AgedBrieItem(Item& item) : item_(item) {}

    void updateQuality() {
        static constexpr int MAX = 50;
        if (item_.quality < MAX) item_.quality++;
        if (item_.sellIn < 1)
            if (item_.quality < MAX) item_.quality++;
    }
};
```

Java 대응 (VSCode / Maven)

```
// Java Step 5-1: AgedBrieItem 생성
// Ctrl+Shift+R → Move to another class
// 또는 새 파일 직접 생성

public class AgedBrieItem {
    private Item item;

    public AgedBrieItem(Item item) {
        this.item = item;
    }

    public void updateQuality() {
        if (item.quality < 50)
            item.quality++;
        if (item.sellIn < 1)
            if (item.quality < 50)
                item.quality++;
    }
}
```

C++: Item& item_ 참조 보관 (Item 클래스 수정 불가 제약 때문) | Java: Item item 필드로 보관 | 클래스 분리 후 반드시 전체 테스트 실행 → 녹색 확인!

C++ (VSCode + clangd)

```
// Step 5-2: BackstagePassItem 클래스 생성
class BackstagePassItem {
    Item& item_;
public:
    explicit BackstagePassItem(Item& item) : item_(item) {}

    void updateQuality() {
        static constexpr int MAX = 50;
        if (item_.quality < MAX) item_.quality++;
        if (item_.sellIn < 11 && item_.quality < MAX) item_.quality++;
        if (item_.sellIn < 6 && item_.quality < MAX) item_.quality++;
        if (item_.sellIn < 1) item_.quality = 0;
    }
};

// Step 5-3: 사용부 (GildedRose.cpp)
void GildedRose::updateQuality() {
    for (auto& item : items) {
        if (item.name == AGED_BRIE) {
            AgedBrieItem{item}.updateQuality(); // 임시 객체 생성
        } else if (item.name == BACKSTAGE_PASS) {
            BackstagePassItem{item}.updateQuality();
        } else if (item.name == SULFURAS) {
            SulfurasItem{item}.updateQuality(); // 아무것도 안 함
        } else {
            NormalItem{item}.updateQuality();
        }
        updateSellIn(item);
    }
}
```

Java 대응 (VSCode / Maven)

```
// Java Step 5-2: 사용부
public void updateQuality() {
    for (Item item : items) {
        if (item.name.equals(AGED_BRIE)){
            new AgedBrieItem(item)
                .updateQuality();
        } else if (item.name.equals(BACKSTAGE)){
            new BackstagePassesItem(item)
                .updateQuality();
        } else if (item.name.equals(SULFURAS)){
            new SulfurasItem(item)
                .updateQuality();
        } else {
            new NormalItem(item)
                .updateQuality();
        }
        updateSellIn(item);
    }
}
```


GildedRoseItem 추상 클래스 생성 → OCP 달성 → Factory 패턴

VSCode 단축키

C++: Ctrl+Shift+R → Extract superclass | Ctrl+. → Create abstract base class

Java: Ctrl+Shift+R → Extract superclass | Alt+Shift+T → Introduce factory

C++ (VSCode + clangd)

```
// Step 6-1: 추상 기반 클래스 GildedRoseItem 생성
// GildedRoseItem.h
#pragma once
#include "Item.h"

class GildedRoseItem {
protected:
    Item& item_;
public:
    explicit GildedRoseItem(Item& item) : item_(item) {}
    virtual void updateQuality() = 0; // 순수 가상 함수 (추상 메서드)
    virtual ~GildedRoseItem() = default;
};
```

Java 대응 (VSCode / Maven)

```
// Java Step 6-1: 추상 클래스
public abstract class GildedRoseItem {
    protected Item item;

    public GildedRoseItem(Item item) {
        this.item = item;
    }
    // Template Method
    public abstract void updateQuality();
}
```

C++ (VSCode + clangd)

```
// Step 6-2: 각 클래스가 GildedRoseItem 상속
class AgedBrieItem : public GildedRoseItem {
public:
    using GildedRoseItem::GildedRoseItem;
    void updateQuality() override {
        if (item_.quality < 50) item_.quality++;
        if (item_.sellIn < 1 && item_.quality < 50) item_.quality++;
    }
};

class BackstagePassItem : public GildedRoseItem {
public:
    using GildedRoseItem::GildedRoseItem;
    void updateQuality() override { /* ... */ }
};

class SulfurasItem : public GildedRoseItem {
public:
    using GildedRoseItem::GildedRoseItem;
    void updateQuality() override { /* 변화 없음 */ }
};

class NormalItem : public GildedRoseItem {
public:
    using GildedRoseItem::GildedRoseItem;
    void updateQuality() override {
        if (item_.quality > 0) item_.quality--;
        if (item_.sellIn < 1 && item_.quality > 0) item_.quality--;
    }
};
```

Java 대응 (VSCode / Maven)

```
// Java Step 6-2: 상속
public class AgedBrieItem
    extends GildedRoseItem {
    public AgedBrieItem(Item item){
        super(item);
    }
    @Override
    public void updateQuality(){
        if(item.quality < 50) item.quality++;
        if(item.sellIn < 1)
            if(item.quality < 50) item.quality++;
    }
}
```

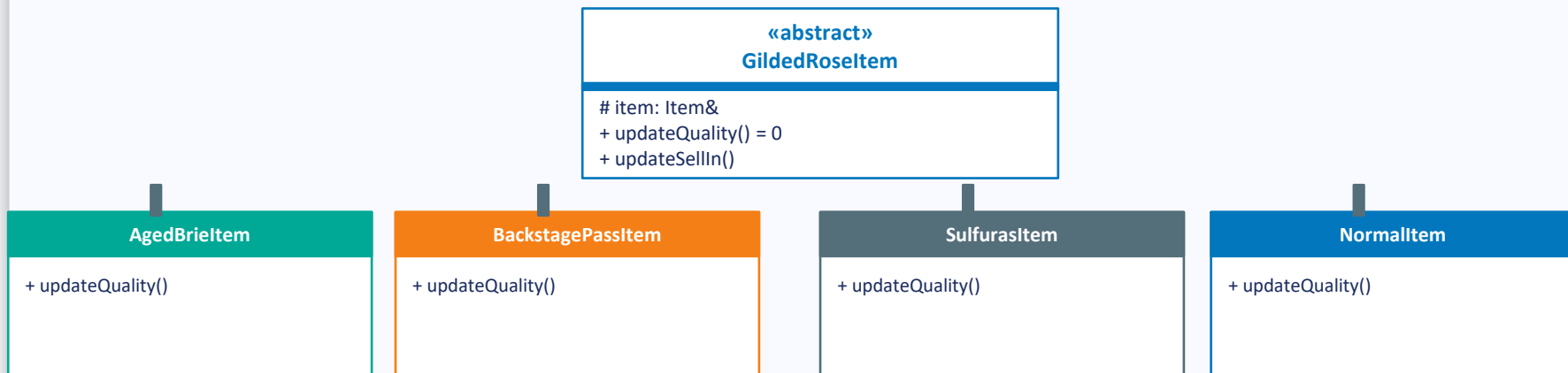
C++ (VSCode + clangd)

```
// Step 6-3: Factory + 다형성으로 깔끔한 updateQuality
std::unique_ptr<GildedRoseItem> createItem(Item& item) {
    if (item.name == AGED_BRIE) return
std::make_unique<AgedBrieItem>(item);
    else if (item.name == BACKSTAGE_PASS) return
std::make_unique<BackstagePassItem>(item);
    else if (item.name == SULFURAS) return
std::make_unique<SulfurasItem>(item);
    else return
std::make_unique<NormalItem>(item);
}
void GildedRose::updateQuality() {
    for (auto& item : items) {
        auto gildedItem = createItem(item); // Factory
        gildedItem->updateQuality(); // 다형성!
        updateSellIn(item);
    }
}
```

Java 대응 (VSCode / Maven)

```
// Java Step 6-3: Factory + 다형성
private GildedRoseItem getItem(Item item){
    if(item.name.equals(AGED_BRIE))
        return new AgedBrieItem(item);
    if(item.name.equals(BACKSTAGE))
        return new BackstagePassesItem(item);
    if(item.name.equals(SULFURAS))
        return new SulfurasItem(item);
    return new NormalItem(item);
}
public void updateQuality() {
    for (Item item : items) {
        GildedRoseItem gi = getItem(item);
        gi.updateQuality(); // 다형성!
        updateSellIn(item);
    }
}
```

클래스 다이어그램 — GildedRose 최종 구조



C++ (VSCode + clangd)

```
// Step 7: 변수명 통일 (gildedRoseItem으로 통일)
// ❌ Before - 각 타입별 다른 변수명
if (item.name == AGED_BRIE) {
    AgedBrieItem agedBrieItem{item}; // 변수명 제각각
    agedBrieItem.updateQuality();
} else if (item.name == BACKSTAGE_PASS) {
    BackstagePassItem backstageItem{item}; // 다른 이름
    backstageItem.updateQuality();
}

// ✅ After - 통일된 변수명 (F2로 일괄 변경)
for (auto& item : items) {
    std::unique_ptr<GildedRoseItem> gildedRoseItem = createItem(item);
    gildedRoseItem->updateQuality(); // 항상 같은 이름
    updateSellIn(item);
}
```

Java 대응 (VSCode / Maven)

```
// Java Step 7: 변수명 통일
// F2 → 새 이름 입력 → 모든 참조 변경

// ❌ Before
GildedRoseItem agedBrieItem = new AgedBrieItem(item);
GildedRoseItem backstageItem = new
BackstagePassesItem(item);

// ✅ After - gildedRoseItem 통일
GildedRoseItem gildedRoseItem;
if(item.name.equals(AGED_BRIE))
    gildedRoseItem = new AgedBrieItem(item);
else if(item.name.equals(BACKSTAGE))
    gildedRoseItem = new BackstagePassesItem(item);
else gildedRoseItem = new NormalItem(item);
gildedRoseItem.updateQuality();
```

VSCode 단축키

C++: Ctrl+R → 이름 바꾸기 | F2 → Rename Symbol (clangd)

Java: F2 → Rename Symbol | Ctrl+F2 → 모든 참조 이름 바꾸기

OCP 검증 — 기존 코드 수정 없이 새 클래스만 추가!

 VSCode 단축키

C++: Ctrl+N → 새 헤더/소스 파일 | Ctrl+. → Create class from usage

Java: Ctrl+N → 새 Java 클래스 | Alt+Insert → Generate Constructor

C++ (VSCode + clangd)

```
// F&B 아이템 요구사항:
// - 일반 아이템보다 2배 빠르게 quality 감소
// - sellIn 지나면 4배 감소 (일반의 2배)

// FoodBeverageItem.h - 새 파일 생성 (기존 파일 수정 없음!)
#pragma once
#include "GildedRoseItem.h"

class FoodBeverageItem : public GildedRoseItem {
public:
    explicit FoodBeverageItem(Item& item)
        : GildedRoseItem(item) {}

    void updateQuality() override {
        // 일반의 2배 감소
        if (item.quality > 0) {
            item.quality--;
            if (item.quality > 0) item.quality--; // 2배
        }
        // sellIn 지나면 추가 2배 (총 4배)
        if (item.sellIn < 1) {
            if (item.quality > 0) {
                item.quality--;
                if (item.quality > 0) item.quality--;
            }
        }
    }
};
```

Java 대응 (VSCode / Maven)

```
// Java - FoodBeverageItem.java (새 파일!)
public class FoodBeverageItem
    extends GildedRoseItem {

    public FoodBeverageItem(Item item) {
        super(item);
    }

    @Override
    public void updateQuality() {
        // 2배 감소
        if (item.quality > 0) {
            item.quality--;
            if (item.quality > 0)
                item.quality--;
        }
        // sellIn 지나면 추가 2배
        if (item.sellIn < 1) {
            if (item.quality > 0) {
                item.quality--;
                if (item.quality > 0)
                    item.quality--;
            }
        }
    }
}
```

C++ (VSCode + clangd)

```
// Factory에 F&B 타입 추가 (GildedRose.cpp - 최소 변경!)
std::unique_ptr<GildedRoseItem> GildedRose::createItem(Item& item) {
    if (item.name == AGED_BRIE) return make_unique<AgedBrieItem>(item);
    else if (item.name == BACKSTAGE_PASS) return make_unique<BackstagePassItem>(item);
    else if (item.name == SULFURAS) return make_unique<SulfurasItem>(item);
    else if (item.name.find("[F&B]") != std::string::npos)
        return make_unique<FoodBeverageItem>(item);
    // ← 추가!
    else
        return make_unique<NormalItem>(item);
}

// F&B 테스트 (FoodBeverageTest.cpp)
TEST(FoodBeverageTest, DegradesTwiceAsNormal) {
    std::vector<Item> items = {Item("[F&B] Bread", 5, 20)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(4, items[0].sellIn);
    EXPECT_EQ(18, items[0].quality); // 20 - 2 = 18
}

TEST(FoodBeverageTest, DegradesFourTimesAfterSellIn) {
    std::vector<Item> items = {Item("[F&B] Milk", 0, 20)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(16, items[0].quality); // 20 - 4 = 16
}

TEST(FoodBeverageTest, QualityNeverBelowZero) {
    std::vector<Item> items = {Item("[F&B] Water", 0, 1)};
    GildedRose app(items);
    app.updateQuality();
    EXPECT_EQ(0, items[0].quality); // 0 유지
}
```

Java 대응 (VSCode / Maven)

```
// Java Factory 업데이트 (최소 변경)
private GildedRoseItem getItem(Item item){
    if(item.name.equals(AGED_BRIE))
        return new AgedBrieItem(item);
    // ... 기존 코드 ...
    if(item.name.contains("[F&B]"))
        return new FoodBeverageItem(item); // 추가
    return new NormalItem(item);
}

// JUnit 5 테스트
@Test void degradesTwice() {
    Item[] items = {
        new Item("[F&B] Bread", 5, 20)};
    new GildedRose(items).updateQuality();
    assertEquals(18, items[0].quality);
}

@Test void qualityNeverNegative() {
    Item[] items = {
        new Item("[F&B] Milk", 0, 1)};
    new GildedRose(items).updateQuality();
    assertEquals(0, items[0].quality);
}
```

8-12 Step 8~12 — 추가 리팩토링 (변수 선언 분리 → 중복 제거 → Inline)

Split Declaration · Remove Duplication · Extract Method · Inline Variable

VSCode 단축키

C++: Ctrl+Shift+R → Inline | Ctrl+. → Inline variable

Java: Ctrl+Shift+R → Inline variable | Alt+Enter → Inline

C++ (VSCode + clangd)

```
// Step 8: 변수 선언 분리 (Split variable declaration)
// ❌ Before - 선언과 초기화 합쳐짐
if (item.name == AGED_BRIE) {
    std::unique_ptr<GildedRoseItem> gi = std::make_unique<AgedBrieItem>(item);
    gi->updateQuality();
} else if (item.name == BACKSTAGE_PASS) {
    std::unique_ptr<GildedRoseItem> gi =
std::make_unique<BackstagePassItem>(item);
    gi->updateQuality();
}

// ✅ After - 선언 분리 후 초기화
std::unique_ptr<GildedRoseItem> gi;
if (item.name == AGED_BRIE) gi = make_unique<AgedBrieItem>(item);
else if (item.name == BACKSTAGE_PASS) gi =
make_unique<BackstagePassItem>(item);
else if (item.name == SULFURAS) gi = make_unique<SulfurasItem>(item);
else gi = make_unique<NormalItem>(item);
gi->updateQuality(); // 선언 1개, 호출 1번!
```

Java 대응 (VSCode / Maven)

```
// Java Step 8: 변수 선언 분리
// Alt+Enter → Split declaration
GildedRoseItem gi;
if(item.name.equals(AGED_BRIE))
    gi = new AgedBrieItem(item);
else if(item.name.equals(BACKSTAGE))
    gi = new BackstagePassesItem(item);
else if(item.name.equals(SULFURAS))
    gi = new SulfurasItem(item);
else
    gi = new NormalItem(item);
gi.updateQuality(); // 호출 1번
```

최종 updateQuality() 는 단 3줄! for 루프 + createltem/getItem + updateQuality 호출 | 깔끔한 코드 = 명확한 의도 표현

8-12 Step 8~12 — 추가 리팩토링 (변수 선언 분리 → 중복 제거 → Inline)

C++ (VSCode + clangd)

```
// Step 9: 중복 코드 제거 (선언 상단, update 하단으로 통합)
void GildedRose::updateQuality() {
    for (auto& item : items) {
        auto gi = createItem(item); // Step 10: Extract Method
        gi->updateQuality();
        updateSellIn(item);
    }
}

// Step 11: Extract Method for factory
std::unique_ptr<GildedRoseItem>
GildedRose::createItem(Item& item) {
    if (item.name == AGED_BRIE)    return make_unique<AgedBrieItem>(item);
    if (item.name == BACKSTAGE_PASS) return
make_unique<BackstagePassItem>(item);
    if (item.name == SULFURAS)    return make_unique<SulfurasItem>(item);
    if (item.name.find("[F&B]") != std::string::npos)
        return
make_unique<FoodBeverageItem>(item);
    return make_unique<NormalItem>(item);
}

// Step 12: Inline variable (임시 변수 제거)
// ❌ Before
auto gi = createItem(item);
gi->updateQuality();

// ✅ After — Inline (Ctrl+Shift+R → Inline variable)
createItem(item)->updateQuality();
```

Java 대응 (VSCode / Maven)

```
// Java Step 9-10: 중복 제거 후 Extract
public void updateQuality() {
    for (Item item : items) {
        updateQuality(item); // Extract!
        updateSellIn(item);
    }
}

private void updateQuality(Item item){
    GildedRoseItem gi = getItem(item);
    gi.updateQuality();
}

private GildedRoseItem getItem(Item item){
    if(item.name.equals(AGED_BRIE))
        return new AgedBrieItem(item);
    if(item.name.equals(BACKSTAGE))
        return new BackstagePassesItem(item);
    if(item.name.equals(SULFURAS))
        return new SulfurasItem(item);
    if(item.name.contains("[F&B]"))
        return new FoodBeverageItem(item);
    return new NormalItem(item);
}

// Java Step 12: Inline variable
// Alt+Enter → Inline variable
// ❌ Before
GildedRoseItem gi = getItem(item);
gi.updateQuality();
// ✅ After
getItem(item).updateQuality();
```

✓ 최종 결과 — Before vs After 비교

✗ Before C++ 레거시

```
// ✗ Before - 100줄의 if 지옥
void updateQuality() {
    for (int i = 0; i < items.size(); i++) {
        if (items[i].name != "Aged Brie"
            && items[i].name != "Backstage passes...") {
            if (items[i].quality > 0)
                if (items[i].name != "Sulfuras...")
                    items[i].quality--;
        } else {
            if (items[i].quality < 50) {
                items[i].quality++;
                if (items[i].name == "Backstage passes...") {
                    if (items[i].sellIn < 11)
                        if (items[i].quality < 50)
                            items[i].quality++;
                }
                // ... 계속 중첩 ...
            }
        }
        // sellIn 로직도 섞여 있음...
    }
}
// 새 아이템 추가? → 전체 if 블록 수정!
// 테스트 작성? → 로직 얽혀 불가능!
```

✓ After C++ 리팩토링 결과

```
// ✓ After - 3줄의 깔끔한 updateQuality
void GildedRose::updateQuality() {
    for (auto& item : items) {
        createItem(item)->updateQuality(); // 다형성!
        updateSellIn(item);
    }
}

// 새 아이템 추가? → 새 클래스만 추가!
class ConjuredItem : public GildedRoseItem {
    void updateQuality() override {
        // 일반의 2배 감소 로직만 작성
        degrade(2);
    }
};

// Factory에 한 줄만 추가:
// if (item.name.contains("Conjured")) return make_unique<ConjuredItem>(item);

// 파일 구조:
// GildedRose.h/cpp      - 핵심 엔진 (수정 불필요)
// GildedRoseItem.h      - 추상 기반 클래스
// AgedBrieItem.h         - Aged Brie 전용 로직
// BackstagePassItem.h    - Backstage 전용 로직
// SulfurasItem.h         - Sulfuras 전용 (빈 구현)
// NormalItem.h           - 일반 아이템
// FoodBeverageItem.h     - F&B (새로 추가)
```

Before vs After 비교

항목	Before (레거시)	After (리팩토링)	개선 효과
코드 길이	updateQuality: 100줄+	updateQuality: 3줄	↓ 97% 감소
새 아이템 추가	전체 if 블록 수정	새 클래스만 추가	OCP 달성
테스트 가능성	통합 테스트만 가능	각 클래스 단위 테스트	테스트 용이
가독성	7단계 중첩 if	명확한 클래스 구조	즉시 이해
확장성	기존 코드 위험 수정	기존 코드 무수정	안전한 확장

VSCode 리팩토링 단축키 cheatsheet — GildedRose 실습용

★ C++ — clangd 기반 리팩토링

F2	이름 바꾸기 (Rename Symbol) — 모든 참조 일괄
Ctrl+Shift+R	리팩토링 메뉴 (Extract Function, Move, Rename...)
Ctrl+.	Quick Fix / Code Action (Extract, Inline, Create...)
Ctrl+Shift+P	CMake: Run Tests CMake: Build
Shift+Alt+F	코드 자동 정렬 (clang-format)
F12	정의로 이동 (Go to Definition)
Shift+F12	모든 참조 찾기 (Find All References)
Ctrl+Shift+`	터미널: cmake --build build && ctest

Java — Extension Pack for Java

F2	이름 바꾸기 (Rename Symbol)
Ctrl+Shift+R	리팩토링 메뉴 (Extract Method, Extract Class...)
Ctrl+.	Quick Fix (Extract, Convert, Generate...)
Ctrl+Shift+P	Java: Run All Tests Maven: clean test
Shift+Alt+F	코드 자동 정렬
F12	정의로 이동
Alt+Enter	IntelliJ 유사 Quick Assist (Extract, Inline...)
Ctrl+Shift+`	터미널: mvn test -q

Git 워크플로우 (각 Step 후 커밋)

git checkout -b refactoring dev → [Step 완료] → git add . → git commit -m 'refactor: Step N 설명' → 모든 테스트 녹색 확인 → 반복

Good Code is not written, it is Refactored.

- ✓ 테스트 없이 리팩토링하지 마세요 — 항상 테스트 먼저
- ✓ 작은 단계로 — 한 번에 하나의 리팩토링만 적용
- ✓ Java와 C++의 OOP는 개념은 같고 문법만 다릅니다
- ✓ VSCode에서도 IntelliJ 못지않은 리팩토링이 가능합니다
- ✓ C++: F2 / Ctrl+Shift+R / Ctrl+. → Java: 동일 단축키

THANK YOU